3GPP TR 38.901 软件库详细设计规格书

第 1 章：核心类库详细规格

本章详述核心类的属性、方法、理论公式及输入输出，严格遵循 3GPP TR 38.901 标准 [1]。

1.1 传播条件模型 (ChannelConditionModel)

1.1.1 抽象基类: ABC_ChannelConditionModel

- **职责**: 定义一个统一接口，用于根据距离和终端高度判断信道的视距（LOS）/非视距（NLOS）状态。
- **方法**:
  - get_condition(self, distance_2d: float, h_ut: float) -> bool:
    - **输入**:
      - distance_2d (float): 基站与用户终端之间的二维水平距离（米）。
      - h_ut (float): 用户终端天线高度（米）。
    - **返回**:
      - is_los (bool): 如果为 LOS 状态，则为 True；否则为 False。
    - **实现逻辑**: 抽象方法，由子类实现具体的概率模型。

1.1.2 具体实现: UMa_ChannelCondition

- **职责**: 实现 UMa 场景下的 LOS 概率判断逻辑。
- **实现**: get_condition 方法
  - **理论公式**: 根据 TR 38.901 表 7.4.2-1 [1]，UMa 场景的 LOS 概率P_{LOS}计算如下：$$P_{LOS} = \begin{cases} 1 & \text{if } d_{2D} \le 18m \\ \left \times \left & \text{if } d_{2D} > 18m \end{cases}$$
    其中，C(d2D,hUT) 定义为：
    C(d2D,hUT)={0(10hUT−13)1.5g(d2D)if hUT<13mif 13m≤hUT≤23m
    而 g(d2D) 定义为：

g(d2D)={045(100d2D)3exp(−150d2D)if d2D≤18mif d2D>18m

- **函数逻辑**:
  1. 根据输入的 distance_2d 和 h_ut 计算 PLOS。
  2. 生成一个在区间内均匀分布的随机数 rand。
  3. 如果 rand < PLOS，返回 True，否则返回 False。

1.2 路径损耗模型 (PathlossModel)

1.2.1 抽象基类: ABC_PathlossModel

- **职责**: 定义一个统一接口，用于计算给定距离和频率下的路径损耗。
- **方法**:
  - calculate_pathloss(self, d_3d: float, d_2d: float, h_bs: float, h_ut: float, is_los: bool, fc_ghz: float) -> float:
    - **输入**:
      - d_3d (float): 三维距离（米）。
      - d_2d (float): 二维距离（米）。
      - h_bs (float): 基站天线高度（米）。
      - h_ut (float): 用户终端天线高度（米）。
      - is_los (bool): LOS 状态。
      - fc_ghz (float): 中心频率（GHz）。
    - **返回**:
      - pathloss_db (float): 路径损耗值（dB）。
    - **实现逻辑**: 抽象方法，由子类实现。

1.2.2 具体实现: UMa_Pathloss

- **职责**: 实现 UMa 场景下的路径损耗计算。
- **实现**: calculate_pathloss 方法
  - **理论公式**: 根据 TR 38.901 表 7.4.1-1 [1]，UMa 场景的路径损耗 PL (dB) 计算如下：
    - **LOS 状态**:
      - 断点距离 dBP′=4hBS′hUT′fc/c，其中$h'*{BS}和 h*{UT}$是有效天线高度，fc 是 Hz 单位的频率，c 是光速。
      - 当 10m≤d2D≤dBP′ 时:

PL1=28.0+22log10(d3D)+20log10(fc)

- 当 dBP′≤d2D≤5km 时:
  $$ PL_2 = 28.0 + 40 \log_{10}(d_{3D}) + 20 \log_{10}(f_c) - 9 \log_{10}((d'{BP})^2 + (h{BS} - h_{UT})^2) $$
- PLUMa−LOS=max(PL1,PL2)

- **NLOS 状态**:
  - PLUMa−NLOS′=13.54+39.08log10(d3D)+20log10(fc)−0.6(hUT−1.5)
  - PLUMa−NLOS=max(PLUMa−LOS,PLUMa−NLOS′)

- **函数逻辑**:
  1. 根据 is_los 标志选择对应的公式分支。
  2. 代入输入参数计算路径损耗。
  3. 返回计算结果。

1.3 大尺度衰落模型 (FadingModel)

1.3.1 抽象基类: ABC_FadingModel

- **职责**: 定义一个统一接口，用于生成一组相互关联的大尺度参数（LSPs）。
- **方法**:
  - generate_lsps(self, is_los: bool) -> LargeScaleParameters:
    - **输入**:
      - is_los (bool): LOS 状态。
    - **返回**:
      - 一个 LargeScaleParameters 数据对象实例。
    - **实现逻辑**: 抽象方法，由子类实现。

1.3.2 具体实现: UMa_Fading

- **职责**: 实现 UMa 场景下的大尺度参数生成。
- **实现**: generate_lsps 方法
  - **理论公式与变量定义**:
    - **输入参数 (来自 ParameterTables)**:
      - LSP 均值向量 μ=T
      - LSP 标准差向量 σ=T
      - LSP 互相关矩阵 C
      - 以上参数均根据 LOS/NLOS 状态从 TR 38.901 表 7.5-6 中获取 [1]。

- **函数逻辑**:
    1. 从 ParameterTables 中根据 is_los 状态获取 UMa 场景的 $\boldsymbol{\mu}$, $\boldsymbol{\sigma}$和$\mathbf{C}$。
    2. 生成一个由 N 个独立的标准正态分布（均值为 0，方差为 1）随机变量组成的向量$\mathbf{z}，其中 N$是 LSP 的数量。
    3. 对互相关矩阵$\mathbf{C}进行**乔列斯基分解(Choleskydecomposition)**，得到下三角矩阵\mathbf{L}，使得\mathbf{C} = \mathbf{L}\mathbf{L}^T$。
    4. 通过矩阵乘法 y=Lz，生成具有正确相关性的随机向量$\mathbf{y}$。
    5. 对向量$\mathbf{y}中的每个元素 y_i$，进行缩放和平移，生成最终的 LSP 值向量$\mathbf{x}$：

       xi=yi·σi+μi
    6. 注意，对于对数正态分布的参数（如 DS, ASA 等），需要进行反对数变换：

       LSPvalue=10xi
    7. 将计算出的所有 LSP 值填充到一个 LargeScaleParameters 对象中并返回。

1.4 架构核心：支持所有 3GPP 场景

本节明确指出，虽然以上章节以**城市宏基站 (UMa)** 场景作为具体实现范例，但整个软件架构的核心是**为支持所有 3GPP TR 38.901 中定义的场景而设计的**。我们通过**策略模式 (Strategy Pattern)** 实现了核心仿真引擎与具体场景实现逻辑的完全解耦。
这意味着，对于每一种计算（如路径损耗、视距判断、衰落参数生成），我们都定义了一个统一的抽象接口。然后，为 TR 38.901 中定义的**每一个场景**，我们都会创建一组实现了这些接口的具体类。当用户在配置文件中指定要仿真的场景时（例如，从'UMa'切换到'UMi-StreetCanyon'），仿真器会自动加载并"插入"对应场景的类模块，而无需改动任何核心代码。
下表清晰地展示了这种"一对多"的映射关系，说明了如何为其他场景（如**城市微基站 (UMi)**、**乡村宏基站 (RMa)**、**室内办公 (InH)**、**室内工厂 (InF)** 和**郊区宏基站 (SMa)**）创建平行的具体实现类[1]。

表 1.1: 场景到具体实现类的映射

场景名称 (配置文件中指定)	信道条件判断类 (实现 ABC_ChannelConditionModel)	路径损耗计算类 (实现 ABC_PathlossModel)	大尺度衰落生成类 (实现 ABC_FadingModel)
'UMa'	UMa_ChannelCondition	UMa_Pathloss	UMa_Fading
'UMi-StreetCanyon'	UMi_ChannelCondition	UMi_Pathloss	UMi_Fading
'RMa'	RMa_ChannelCondition	RMa_Pathloss	RMa_Fading

'InH-Office'	InH_ChannelCondition	InH_Pathloss	InH_Fading
'InF-...'	InF_ChannelCondition	InF_Pathloss	InF_Fading
'SMa'	SMa_ChannelCondition	SMa_Pathloss	SMa_Fading

每个具体实现类（如 RMa_Pathloss）的内部逻辑将严格遵循 TR 38.901 中为其对应场景定义的特定公式和参数表 [1]。这种设计确保了软件库的
模块化、高可扩展性和标准遵从性。

1.5 仿真引擎 (ChannelGenerator)

- **职责**: 编排并执行完整的 12 步信道系数生成流程。
- **核心方法**: generate_channel_realization()
  - **实现**: 严格按照 TR 38.901 第 7.5 节的 12 个步骤执行 [1]。以下是关键步骤的公式实现：
    - **步骤 5: 生成簇延迟**
      - **公式 (7.5-1)**: τn′=−rτ·σDS·ln(Xn)，其中 Xn~U(0,1)，$r_{\tau}和\sigma_{DS}$来自参数表。
    - **步骤 6: 生成簇功率**
      - **公式 (7.5-5)**: Pn′=exp(−τn′rτσDSrτ−1)·10−Zn/10，其中 Zn 是簇阴影衰落。
      - **LOS 功率 (7.5-7, 7.5-8)**: P1,LOS=KR+1KR，Pn,diff=KR+11Pn。
    - **步骤 7: 生成角度**
      - **方位角 (AOA/AOD, 公式 7.5-9)**: 使用逆高斯函数，并结合 LOS 方向和随机偏移生成。
      - **天顶角 (ZOA/ZOD, 公式 7.5-14)**: 使用逆拉普拉斯函数，并结合 LOS 方向和随机偏移生成。
    - **步骤 11: 生成信道系数**
      - **公式 (7.5-22)**: 这是核心的合成公式，对每个天线对$(u,s)和每个簇 n$进行计算。
        Hu,s,n(t)=MPnm=1ΣM[Frx,u,θ(θn,m,ZOA,φn,m,AOA)Frx,u,φ(θn,m,ZOA,φn,m,AOA)]TejΦn,mθθκn,m−1ejΦn,mφθκn,m−1ejΦn,mθφejΦn,mφφ[Ftx,s,θ(θn,m,ZOD,φn,m,AOD)Ftx,s,φ(θn,m,ZOD,φn,m,AOD)]ejλ02π(r^rx,n,mTdrx,u)ejλ02π(r^tx,n,mTdtx,s)ejλ02π(r^rx,n,mTv)t
      - **变量定义**:
        - Pn: 第 n 簇的功率。
        - M: 每簇的径数（通常为 20）。
        - Frx,u,θ/φ,Ftx,s,θ/φ: 收发天线单元 u,s 在$\theta 和\phi$极化方向

上的场方向图[1]。
- Φn,m: 第 n 簇第 m 径的随机初始相位。
- κn,m: 交叉极化功率比。
- r^rx,n,m,r^tx,n,m: 到达和离开方向的单位球面向量。
- drx,u,dtx,s: 收发天线单元 u,s 的位置向量。
- v: 用户终端的速度向量。
- λ0: 载波波长。

第 2 章：用户图形界面 (GUI) 设计概念

一个直观的 GUI 可以极大地简化仿真配置和结果分析。以下是一个基于 Python 的 GUI 框架（如 PyQt 或 Tkinter）的设计概念。

2.1 界面布局

主窗口将分为三个主要区域：

1. **配置面板 (左侧)**: 用于用户输入所有仿真参数。
2. **可视化面板 (中间)**: 用于以图形方式显示仿真结果。
3. **日志与输出面板 (底部)**: 显示仿真进度、关键输出参数和任何错误信息。

2.2 配置面板控件

- **场景选择**:
  - **控件**: 下拉菜单 (Dropdown/ComboBox)。
  - **选项**: 'UMa', 'UMi-StreetCanyon', 'RMa', 'InH-Office', 'InF' 等。
- **频率与带宽**:
  - **控件**: 文本输入框 (LineEdit/Entry) 带单位标签。
  - **输入**: 中心频率 (GHz)，带宽 (MHz)。
- **网络布局**:
  - **控件**: 文件加载按钮 ("Load BS/UE Positions")。
  - **功能**: 允许用户加载一个 CSV 或 JSON 文件，其中包含基站和用户终端的 3D 坐标及速度向量。
- **天线配置**:
  - **控件**: 独立的配置窗口或一组文本框。
  - **输入**: 天线阵列维度 (M, N, P)、单元间距 (dH,dV)、极化模型等[1]。
- **仿真控制**:

- **控件**: "Run Simulation" 按钮和进度条。

2.3 用户工作流程

1. 用户在配置面板中选择场景、输入频率等参数。
2. 用户加载网络布局文件。
3. 用户点击 "Run Simulation" 按钮。
4. GUI 调用 ChannelSimulator 外观类的__init__和 run_simulation 方法。
5. 仿真过程中，进度条更新，日志面板显示当前执行的步骤。
6. 仿真完成后，结果会自动在可视化面板中呈现。

第 3 章：仿真结果呈现

有效的可视化是理解信道特性的关键。GUI 应提供多种方式来呈现 ChannelRealization 对象中的数据。

3.1 表格化呈现

在日志/输出面板的一个标签页中，以表格形式清晰地列出每个链路计算出的关键大尺度参数：

链路 (BS-UE)	LOS/NLOS	路径损耗 (dB)	延迟扩展 (ns)	K 因子 (dB)	角度扩展 (ASD, ASA, ZSD, ZSA)
(1, 1)	NLOS	125.4	150.2	N/A	(15.2, 45.8, 5.1, 8.3)
(1, 2)	LOS	98.7	35.6	9.1	(5.5, 12.1, 2.3, 4.0)

3.2 图形化呈现

可视化面板应包含多个标签页，每个标签页展示一种图形：

- **功率延迟曲线 (Power Delay Profile - PDP)**:
  - **X 轴**: 延迟 (ns)
  - **Y 轴**: 归一化功率 (dB)
  - **描述**: 直观展示多径分量的到达时间和相对强度，帮助理解信道的时间色散特性。
- **角度功率谱 (Angular Power Spectrum - APS)**:

    - **图 1 (离开角)**:
        - **X 轴**: 方位角 (AOD, degrees)
        - **Y 轴**: 天顶角 (ZOD, degrees)
        - **颜色/大小**: 功率 (dB)
    - **图 2 (到达角)**:
        - **X 轴**: 方位角 (AOA, degrees)
        - **Y 轴**: 天顶角 (ZOA, degrees)
        - **颜色/大小**: 功率 (dB)
    - **描述**: 以 2D 散点图的形式展示各个簇在空间角度上的分布和功率，帮助理解信道的空间选择性。
- **信道脉冲响应 (Channel Impulse Response - CIR)**:
    - **X 轴**: 时间 (ns)
    - **Y 轴**: 幅度
    - **描述**: 显示合成后的信道冲激响应的幅值，是信道在时域的完整表征。
- **网络拓扑与射线可视化**:
    - **类型**: 2D 或 3D 场景图。
    - **内容**: 绘制基站和用户终端的位置。可以选择一条链路，并以线条形式可视化该链路的主要簇（或射线）的离开和到达方向，提供一个几何直觉。

Works cited

1. 38901-j00.docx